Journal of Systems Architecture 147 (2024) 103051

Contents lists available at ScienceDirect

Journal of Systems Architecture

journal homepage: www.elsevier.com/locate/sysarc

PED: Probabilistic Energy-efficient Deadline-aware scheduler for heterogeneous SoCs

Xing Chen [a,*], Anish Krishnakumar [b], Umit Ogras [b], Chaitali Chakrabarti [a]

[a] School of Electrical, Computer and Energy Engineering, Arizona State University, United States
[b] Department of Electrical and Computer Engineering, University of Wisconsin-Madison, United States

ARTICLE INFO

Keywords:
Scheduling
Energy-efficient
Soft deadline
Heterogeneous SoC
Domain-specific SoC

ABSTRACT

Heterogeneous systems-on-chip (SoCs) integrate diverse cores with different performance and energy tradeoffs. Scheduling applications with soft deadline constraints is highly complex in such heterogeneous platforms, and the complexity is further exacerbated by the streaming jobs generated by applications from domains such as communication and radar systems. Existing deadline-aware schedulers typically first translate the job deadlines to task-level slacks before scheduling, which is the time available for a processing element (PE) to execute a specific task. Task-level slacks are critically dependent on the task-to-PE allocation of all other tasks from the same job (intra-job) or concurrent jobs (inter-job). However, this allocation is usually unknown before the start of the scheduling process. To address the problem, we propose *PED*, a probabilistic energy-efficient deadline-aware scheduler for heterogeneous SoCs. *PED* minimizes the average tardiness of streaming jobs with the least energy consumption by accurately predicting the task-to-PE allocation using Neural Network and considering intra- and inter-job contentions when scheduling tasks. Our extensive experimental results in a domain-specific SoC (DSSoC) designed for radar and communication domains show that *PED* can reduce tardiness by 6.9× with comparable energy consumption; and reduce energy consumption by 14% without any loss in tardiness, when compared with state-of-the-art schedulers.

1. Introduction

Heterogeneous systems-on-chip (SoCs) bring about substantial improvements in performance and energy efficiency compared to homogeneous counterparts by leveraging the strengths of diverse PEs [1,2]. As a distinctive class of heterogeneous SoCs, domain-specific SoCs or DSSoCs, intelligently integrate general-purpose cores and hardware accelerators to optimize performance and energy efficiency for specific application domains [3–5]. Tailoring DSSoCs for a specific domain allows designers to exploit their full potential to maximize performance and energy efficiency.

Time-sensitive applications, such as those in communication [6], radar [7,8], multimedia [9], and HPC [10,11], should satisfy their soft deadlines as much as possible. The soft deadline does not have to be met strictly (in contrast to the hard deadline) but usually comes with a penalty. The penalty is associated with utility value of the results [9], quality of service (QoS) [11], and monetary profit [10], etc. In this paper, the penalty is measured by tardiness [11,12]. An intuitive way to minimize tardiness is to run jobs on the PEs with the earliest finish time such that the jobs are completed as fast as possible, but this could result in unnecessarily high energy consumption. On the other hand, when the deadline is long, the jobs could have run on more energy-efficient PEs and still meet the deadline with lower energy consumption. Designing a scheduling algorithm that minimizes both tardiness and energy consumption is difficult. The difficulty is significant for streaming applications, such as in radar and communication, where multiple jobs have to run on the DSSoC simultaneously.

Furthermore, the jobs in radar and communication domains such as WiFi-TX/RX and Temporal Mitigation, consist of multiple dependent tasks, which are modeled by the directed acyclic graph (DAG). Each node in DAG represents a task. Soft deadline constraints are specified per application, but scheduling is performed at the task granularity level, which is each node in a DAG [1,13]. Therefore, application (or) job-level deadlines must be translated to task-level slacks, which is a measure of the time available in a PE to execute a particular task. Schedulers attempt to assign tasks to PEs that can complete their execution within the available slack.

Task-level slack on a PE critically depends on the tasks that are currently executed on the PE and the unallocated tasks which will be executed on the PE in the future. The unallocated tasks compete for the same set of PEs, resulting in a decrease in the task-level slacks. In

* Corresponding author.
E-mail address: xchen382@asu.edu (X. Chen).

https://doi.org/10.1016/j.sysarc.2023.103051
Received 25 July 2023; Received in revised form 31 October 2023; Accepted 7 December 2023
Available online 9 December 2023
1383-7621/© 2023 Elsevier B.V. All rights reserved.

the case when multiple jobs have to run simultaneously, the scheduler must be aware of the contentions from unallocated tasks within the job and across other concurrently executing jobs. These contentions can be determined only if the task-to-PE allocation of the unallocated tasks can be predicted accurately. State-of-the-art works assess the contention based on highly simplified assumptions. Examples include modeling contention using the shortest execution time [1] or the average execution time across PEs [13–15], resulting in unnecessary tardiness or high energy consumption. Prior knowledge of domain applications in DSSoCs, including the DAG structure, power consumption, and execution time, allows us to exploit their behavior to build an accurate model to predict the allocation probability of unallocated tasks accurately.

This paper proposes a **P**robabilistic **E**nergy-efficient **D**eadline-aware (*PED*) scheduler for heterogeneous DSSoCs that minimizes tardiness with the lowest energy consumption for streaming applications with soft deadline constraints. *PED* computes the probability of different task-to-PE allocations using a neural network (NN)-based pre-allocation step. The allocation probabilities are used to compute the PE occupancies due to intra- and inter-job contentions. The job-level slack is derived from the soft deadline constraints by taking the inter-job contentions into account. Then, the task-level slack is derived from the job-level slack by considering intra-job contentions. The major contributions of this paper are as follows:

- A probabilistic energy-efficient deadline-aware scheduler *PED* that minimizes tardiness with the least energy consumption for streaming jobs.
- An NN-based technique to estimate the allocation probability of tasks and use the probability to assess the contentions due to allocated tasks.
- Using the inter-job and intra-job contentions to derive a more accurate task-level slack.
- Validation of the *PED* scheduler performance w.r.t. tardiness and energy consumption for realistic applications.

The performance of *PED* is extensively evaluated and compared with (i) the earliest deadline first (*EDF*) scheduler [16], (ii) a static power-aware scheduler (*NDES*) [1], and (iii) two dynamic power-aware list-based schedulers (*PL* [13] and *PECS* [17]). On average, *PED* outperforms *EDF* and *PL* schedulers by achieving 22% and 18% lower energy consumption with a constant frequency setting and 19% and 14% with a deadline-aware dynamic voltage-frequency scaling (DVFS) policy without any loss in tardiness, respectively. When compared with *NDES* and *PECS* schedulers with a constant frequency setting, *PED* reduces the tardiness by 6.9× and >20× on average, respectively.

The rest of the paper is organized as follows. Section 2 discusses the relevant work and prior approaches. Section 3 presents the system model and the problem definition. The proposed *PED* scheduler is presented in Section 4 followed by the experimental results in Section 5. Section 6 concludes the paper.

2. Related work

Task scheduling algorithms play a crucial role in reducing the job makespan or energy consumption in an SoC by allocating the tasks to the most suitable PEs [18,19]. Task scheduling in heterogeneous SoCs is a highly challenging problem since it is NP-complete [20]. The challenge is even larger for heterogeneous SoCs when multiple jobs execute simultaneously. Heuristic schedulers are extensively used because of the lower complexity when compared to optimization-based approaches. For example, heterogeneous earliest finish time (HEFT) [19], predict earliest finish time (PEFT) [21] and heterogeneous makespan minimizing DAG scheduler (HMDS) [22] are heuristic algorithms proposed to minimize the makespan of a single DAG. For dynamic multi-DAG case, earliest finish time first (ETF) [23] and other HEFT-based heuristics [24,25] minimize the job makespan, and *EDF* scheduler minimizes the deadline violation rate [16].

Energy-efficient schedulers with deadline constraints strive to minimize energy consumption while satisfying job-level deadlines [13,17,26]. Many papers have investigated the problem when scheduling independent tasks [35–42]. However, the scheduling methods for independent tasks cannot be utilized when tasks have dependencies since the intra-job contention from dependent tasks has to be considered. The scheduling techniques for dependent tasks represented by DAGs can be classified into two main categories based on how energy minimization is achieved: (i) use scheduling algorithms to meet deadlines and use DVFS policies to minimize energy consumption, and (ii) use scheduling algorithms that simultaneously meet deadlines and minimize energy consumption. Table 1 summarizes the key features of the related deadline-aware and energy-efficient works. Schedulers in [26,27] first run task schedulers to minimize the makespan and then lower the frequency to fill in the idle times of PE. However, when the makespan of jobs is short (<1 ms) compared to the DVFS granularity (>10 ms), it is impractical to fill in the slacks using DVFS based approaches. Other schedulers migrate the tasks out of minimally utilized PEs so that these PEs can be power-gated and their energy consumption reduced [18].

There are still other approaches that use task scheduling algorithms to simultaneously meet job-level deadlines while minimizing energy consumption by improving the utilization of energy-efficient PEs. Some prior works [1,28] schedule tasks in an iterative manner, where tasks are allocated to the slowest PEs to save energy initially and iteratively moved to faster PEs to meet the deadlines. Other works translate job-level deadlines into task-level slacks and assign tasks to PEs with sufficient slack [13–15,17,29–31]. However, task-level slack can only be accurately determined if the task-to-PE allocation for the unallocated tasks is predicted accurately. Thus, it is important to derive a technique to better predict the task-to-PE allocation.

The task-to-PE allocation depends on many factors, such as the execution time of tasks in PEs, the availability of PEs, and the position of tasks in the job DAG. Existing schedulers make simple assumptions on the task-to-PE allocation, resulting in poor performance in saving energy and meeting deadlines. The works in [13,14] estimate the contentions using the average execution time of unallocated tasks across all PEs, based on the assumption that tasks are allocated to all PEs with equal probability. The schedulers in [17,29,43], and [30] estimate the task-to-PE allocation simply based on the execution time of tasks. In contrast, the proposed *PED* scheduler utilizes an NN-based method to predict the task-to-PE allocation using features describing both tasks and SoC, resulting in better scheduling performance.

In the case of streaming jobs where multiple jobs execute in parallel, the tasks in multiple jobs compete with each other for the same set of PEs. Hence it is necessary to consider inter-job contentions of unallocated tasks from other jobs when estimating the task-level slacks. However, the scheduling algorithms using task-slack such as Safari and Khorsand [13], Senapati et al. [14], Tian et al. [17], Xie et al. [29] and Arabnejad et al. [15] focus on a single job executing in the system and thus do not consider inter-job contentions due to multiple simultaneously executing jobs. The work in [33] computes the inter-job contention based on the static optimal allocation of a single job, and ignores the fact that static allocation is no longer the best when the PE availability changes. Some works reschedule all tasks in system when a new job arrives, as in [32,34], introducing high complexity. The risk-aware scheduling work in [44] estimates the probability of task-to-PE allocations to assess the contentions, which is similar to our method. However, the method in [44] requires experts to manually design the allocation function.

Our proposed *PED* scheduler explicitly considers the effect of both the inter-job and intra-job contentions and uses an NN to estimate the task-to-PE allocations accurately.

Table 1
Summary of the state-of-the-art energy-efficient and deadline-aware works.

Reference	Scheduling to minimize energy	Inter-job contention	Intra-job contention	Metrics
EES [26]	✗	✗	✓	Energy
CEAS [27]	✗	✗	✓	Energy
DEWTS [18]	✗	✗	✓	Energy
NDES [1]	✓	✗	✓	Energy
DNDS [28]	✓	✗	✓	Energy
DUECM [29]	✓	✗	✓	Energy
PESTRO [14]	✓	✗	✓	Energy and makespan
PECS [17]	✓	✗	✓	Energy and EDP
MinD+ED [30]	✓	✗	✓	Energy and tardiness
PL [13]	✓	✗	✓	Deadline miss rate and energy
BLCBS [31]	✓	✗	✓	Energy
LAS [32]	✓	✓	✗	Energy and deadline miss rate
TETRIS [33]	✓	✓	✗	Energy and makespan
MMKP-MDF [34]	✓	✓	✗	Energy and deadline miss rate
PED (proposed)	✓	✓	✓	Energy and tardiness

Table 2
Summary of the primary notations used in this paper.

Notation	Definition
t_i	Task i
PE_m	Processing element m
$w_{i,m}$	Execution time of t_i in PE_m
$p_{i,m}$	Allocation probability of allocating t_i to PE_m
$\tilde{w}_{i,m}$	Occupancy time for t_i in PE_m using allocation probability
$\boldsymbol{T}_k$	The kth job
$\boldsymbol{T}_k^{prior}$	Jobs that came in earlier than job $\boldsymbol{T}_k$
$\boldsymbol{T}_k(i)$	The set of unallocated tasks in job $\boldsymbol{T}_k$ when allocating t_i
$\boldsymbol{T}_k^{critical}(i)$	Critical child tasks of t_i
$\boldsymbol{T}_k^{non}(i)$	Non-critical tasks of t_i
D_k^{soft}	Soft job-level deadline of job $\boldsymbol{T}_k$
$SLACK_{k,m}$	Job-level slack of job $\boldsymbol{T}_k$ in PE_m
$slack_{i,m}$	Task-level slack of task t_i in PE_m

3. System model and problem definition

3.1. Job model

This paper considers streaming jobs with random arrival times, where each job has a soft deadline constraint. Thus, at any given time instant, there may be multiple parallel jobs in the system. The kth job is represented by DAG, $(\boldsymbol{T}_k, \boldsymbol{E}_k)$, where $\boldsymbol{T}_k$ is the set of nodes representing the tasks in the job and $\boldsymbol{E}_k$ denotes the set of edges, where edge $e_{i,j} \in \boldsymbol{E}_k$ represents the dependency between task t_i (parent) and task t_j (child). Edge $e_{i,j}$ is annotated with the amount of data transmitted from t_i to t_j. The communication time between t_i and t_j is 0 if t_i and t_j are executed in the same PE, otherwise, it depends on the bandwidth between PEs.

The execution time of task t_i running on PE_m is given by $w_{i,m}$. Tasks without parents are entry tasks, and tasks without children are exit tasks.[1] If the arrival time of job $\boldsymbol{T}_k$ is τ_k, then $L(\boldsymbol{T}_k)$, the makespan of $\boldsymbol{T}_k$, is defined as the difference between τ_k and the finish time (FT) of exit task $t_{exit} \in \boldsymbol{T}_k$:

$$L(\boldsymbol{T}_k) = FT(t_{exit}) - \tau_k \tag{1}$$

The soft deadline of $\boldsymbol{T}_k$ is D_k^{soft}. The tardiness of a job indicates the time difference between the makespan $L(\boldsymbol{T}_k)$ and soft deadline [12], which can be computed as follows:

$$Tardiness_k = max(L(\boldsymbol{T}_k) - D_k^{soft}, 0) \tag{2}$$

If a job is completed before the soft deadline D_k^{soft}, the tardiness is 0. Table 2 summarizes the notations used in this paper.

[1] If there are multiple entry/exit tasks, an additional entry/exit task with 0 latency is added to the original multiple entry/exit tasks in the DAG.

3.2. SoC and energy model

The domain-specific heterogeneous SoC under consideration is a clustered multi-core system consisting of M PEs, grouped into N clusters. Each cluster consists of one or more identical PEs, which operate at the same frequency/voltage settings. For instance, the DSSoC used in our later experiments has five clusters, where the four 'LITTLE' cores form a cluster, the four 'big' cores form another cluster, and three types of accelerators correspond to three more clusters. While the execution time and energy of a task are the same for all PEs in a cluster, they differ from cluster to cluster. The general-purpose PEs (i.e. Arm big.LITTLE cores) can run all tasks, while the accelerators can only run specific tasks.

The total energy consumption consists of three parts:

- $E^{SoC-periph}$: the energy from peripheral circuits of the SoC,
- E^{idle}: the idle energy consumption of the PEs,
- E^{active}: the energy consumption when the PEs are actively executing tasks.

Hence, the overall energy consumption of SoC is calculated using Eq. (9) as:

$$\begin{aligned} E &= E^{SoC-periph} + (E^{idle} + E^{active}) \\ &= (P^{SoC-periph} \cdot L) + (\sum_{m\in[1,M]} P_m^{static} \cdot L_m^{idle}) \\ &+ (\sum_{m\in[1,M]} (P_m^{static} + P_m^{dynamic}) \cdot L_m^{active}) \end{aligned} \tag{3}$$

where E is the overall energy consumption of K jobs, $P^{SoC-periph}$ is a constant used to model the power of peripheral circuits. L is the time to finish K jobs, defined by the difference between the arrival time of the first job and the finish time of the last job. L_m^{active} and L_m^{idle} are the active and idle time period of PE_m, respectively. P_m^{static} is the leakage power and $P_m^{dynamic}$ is the dynamic power of PE_m. The dynamic power consumption increases quadratically with voltage and the frequency increases approximately linearly with voltage [45]:

$$P_m^{dynamic} = \alpha C V_m^2 f_m \tag{4}$$

where α is the activity factor, C is the load capacitance, and V_m and f_m are the voltage and frequency settings of PE_m.

3.3. Problem definition

In this paper, we consider randomly streaming jobs from domain-specific applications, where the DAG representation of a job is known but the arrival time of a job is not known beforehand. We assume that the dynamic power of PEs, and the time and energy to execute a task in each PE, are known beforehand. If they are not available, light-weight tools, such as simple perf [46] and PAPI [47], can be used to learn the

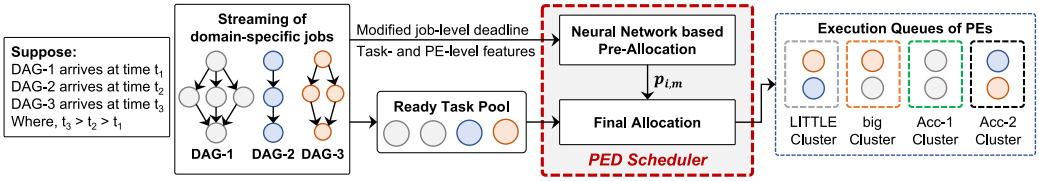


Fig. 1. An overview of the workflow of the proposed *PED* scheduler that consists of two key steps: (i) pre-allocation and (ii) final-allocation. The circles with different colors represent the tasks from different jobs.

power-performance of the tasks on different PEs, similar to Gupta et al. [48].

Our objective is to find a non-preemptive schedule S for K streaming jobs that minimizes the average tardiness (primary) while consuming the lowest energy E (secondary). We assume that the K streaming jobs $\{\boldsymbol{T}_1, \ldots, \boldsymbol{T}_K\}$ arrive at random times $\{\tau_1, \ldots, \tau_K\}$ with each job having a soft deadline constraint $\{D_1^{soft}, \ldots, D_K^{soft}\}$. For a fair comparison of jobs with different soft deadlines, we normalize the *Tardiness* of a job $\boldsymbol{T}_k$ to its soft deadline. Thus, the average normalized tardiness of the K jobs is defined as:

$$Avg.\ Tardiness = \frac{1}{K}\sum_{k=1}^{K}\frac{Tardiness_k}{D_k^{soft}} \tag{5}$$

Hence, the objective function can be written as:

$$\min_{S\in S_T} E[\arg\min_{S_T} Avg.\ Tardiness] \tag{6}$$

where $E[S]$ is the energy consumption of schedule S where each schedule S is defined by the task-to-PE allocation and the start time of each task. S_T is a set of schedules that achieve the minimum tardiness. A feasible schedule should meet the following three constraints:

- *Non-preemptive Constraint.* A task can only be executed by one PE, i.e., the execution of a task cannot be shared by multiple PEs.
- *Dependency Constraint.* The start time of a task t_i must be greater or equal to the summation of the finish time of task t_j and the communication time from t_i to t_j, for any given predecessor t_j.
- *Resource Constraint.* A PE can execute at most one task at a given time.

4. Overview of the proposed *PED* scheduler

This section introduces the proposed *PED* scheduler, outlined in Fig. 1. It consists of 2 steps: (i) NN-based pre-allocation and (ii) final-allocation. The pre-allocation step estimates the probability of task-to-PE allocations for all tasks in a newly arrived job. We utilize an offline-trained NN to compute the probability of allocating a task to each of the clusters and then transform the task–cluster allocation probabilities to task-to-PE allocation probabilities (Fig. 2). This two-step approach helps reduce the size of NN used to derive the task–cluster allocation probability. Next, the task-to-PE allocation probability is transformed to the expected occupancy time of the task in the PE. The PE occupancy times are first used to derive the job-level slack in different PEs (denoted as $SLACK_{k,m}$), which is the time available in PE_m to execute tasks in job $\mathbf{T}_k$ by considering the inter-job contentions. The job-level slack is then translated to task-level slacks for that PE by considering the intra-job contentions. Finally, the task is allocated to a PE such that the task-level slack is met with the least energy consumption.

In this section, we start with the inter-job and intra-job contentions in Section 4.1, then introduce the NN-based pre-allocation in Section 4.2. Section 4.3 introduces the final-allocation step including the detailed computation of job-level and task-level slack.

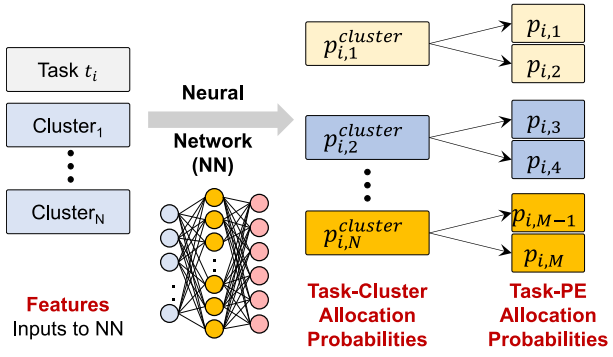


Fig. 2. Pre-allocation: NN generates the task–cluster allocation probability, which is then transformed to task-to-PE probability by scaling it by the normalized job-level slack.

4.1. Inter-job and intra-job contentions

When multiple parallel jobs have to be executed simultaneously, the scheduler should take into account the contentions due to unallocated tasks in the same job (intra-job), and across jobs (inter-job):

- *Inter-job* contention is due to *other* jobs competing for the same set of PEs. It reduces the job-level slack, the time available in a PE to execute the tasks in a particular job. The job-level slack $SLACK_{k,m}$ in PE_m is derived by subtracting out the time to complete the tasks that have been allocated earlier, and the time set aside for unallocated tasks from other jobs from the soft deadline D_k^{soft}. The detailed derivation is given in Section 4.3.
- *Intra-job* contention is due to the unallocated tasks in the same job competing for the same set of PEs. It also reduces the time available in PEs to execute a particular task. We use the task-level slack $slack_{i,m}$ to represent the time available in PE_m for task t_i considering the intra-job contentions. Details are provided in Section 4.3.

To account for the inter-job and intra-job contentions, we need to predict the task-to-PE allocation of unallocated tasks accurately. Consider the example in Fig. 3, where the job DAG shown in Fig. 3(a) has to be allocated to two PEs. The tasks in the DAG are identical, and the execution time and energy consumption to run the task in PE_1 is 1s and 5J, and in PE_2 is 2s and 1J. To schedule t_0 in $\mathbf{T}_1$, it is necessary to first predict the task-to-PE allocation of the unallocated tasks t_1 to t_4. If the scheduler greedily assumes all unallocated tasks will be allocated to PE_1, t_0 will be allocated to PE_2 to minimize energy consumption since both PE_1 and PE_2 have enough time to execute t_0 (Fig. 3(b)). If the scheduler assumes unallocated tasks will be allocated to both PEs evenly, t_0 will still be allocated to PE_2 (Fig. 3(c)). However, both schedules result in higher energy consumption compared with the optimal schedule where t_0 is allocated to PE_1, as shown in Fig. 3(d). To achieve the optimal schedule, the scheduler should be able to predict that t_1 to t_4 will be allocated to PE_2. Next, consider the case when there is a concurrent job $\mathbf{T}_0$ running in the SoC simultaneously. $\mathbf{T}_0$

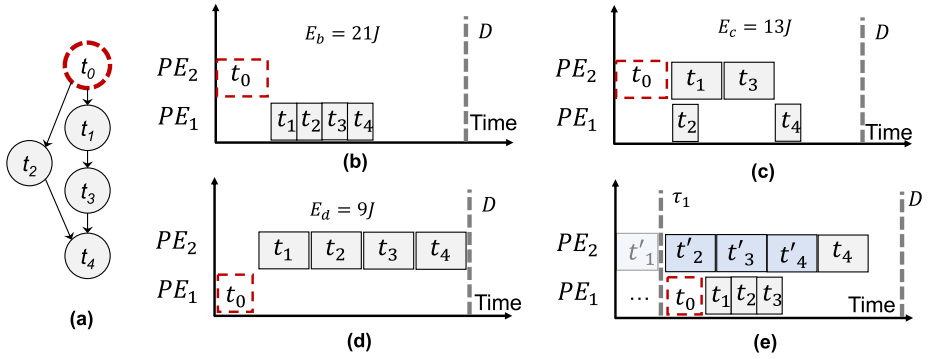


Fig. 3. (a) DAG of job $\mathbf{T}_1$, where task t_0 is to be scheduled. (b) Schedule with the assumption that all unallocated tasks will be allocated to PE_1, (c) Schedule with the assumption that unallocated tasks will be allocated to both PEs evenly; (d) Optimal schedule, when SoC executes a single job without inter-job contentions. (e) Optimal schedule with inter-job contentions due to tasks in concurrent job (blue box).

has the same DAG as $\mathbf{T}_1$ but arrived earlier. When $\mathbf{T}_1$ arrives at time τ_1, tasks t'_0 and t'_1 in job $\mathbf{T}_0$ have already been completed and the task-to-PE allocations of task t'_2, t'_3 and t'_4 have been predicted. Since the unallocated tasks in job $\mathbf{T}_0$ compete with tasks in $\mathbf{T}_1$ for PEs, the optimal schedule of $\mathbf{T}_1$ changes as shown in Fig. 3(e).

The PE occupancy due to the unallocated tasks can be estimated using the allocation probabilities generated by the NN in the pre-allocation step. Let $p_{i,m}$ denote the probability of allocating t_i to PE_m. We transform the probability to the expected occupancy time of t_i in PE_m using the product of the execution time and allocation probability:

$$\tilde{w}_{i,m} = p_{i,m} \cdot w_{i,m} \tag{7}$$

where $w_{i,m}$ is the execution time of task t_i running in PE_m.

4.2. Pre-allocation based on neural network

When a job with K tasks arrives, the pre-allocation step in *PED* scheduler is evoked K times to compute the allocation probability of all tasks in the job. The allocation probability is then used to compute the expected occupancy time in different PEs as shown in Eq. (7). In this section, we describe the highly optimized NN that is used to calculate the allocation probability (Fig. 2).

PED uses a tiny fully connected neural network with only one hidden layer to predict the allocation probability. The input of NN is the feature vector set $\boldsymbol{X}_i$ for task $t_i \in \boldsymbol{T}_k$, and the output is a vector with N values representing the probability of allocating task t_i to N clusters. We create an NN which generates task–cluster allocation probabilities since it has low overhead compared to an NN which generates task-to-PE allocation probabilities.

$$\boldsymbol{p}_i^{cluster} = [p_{i,1}^{cluster}, ..p_{i,n}^{cluster}., p_{i,N}^{cluster}] = f(\boldsymbol{X}_i) \tag{8}$$

where $f(\cdot)$ is the NN function and $p_{i,n}^{cluster}$ is the probability of allocating t_i to $cluster_n$. Intuitively, the PE with more time available to execute tasks in a job should have a higher allocation probability. As described in Section 4.1, the job-level slack represents the time available to run job $\mathbf{T}_k$ considering the inter-job contentions. Thus the task-to-PE allocation probability $p_{i,m}$ can be computed by scaling the task–cluster allocation probability with the normalized job-level slack in each PE:

$$\begin{aligned} p_{i,m} &= \frac{\text{job-level slack in PE}_m}{\text{job-level slack in cluster}_n} \cdot p_{i,n}^{cluster} \\ &= \frac{SLACK_{k,m}}{\sum_{\text{PE}_m \in cluster_n} SLACK_{k,m}} \cdot p_{i,n}^{cluster} \end{aligned} \tag{9}$$

where the $SLACK_{k,m}$ is the job-level slack in PE_m; the detailed computation of $SLACK_{k,m}$ is given in Section 4.3.

4.2.1. Features of neural network

We start with a NN using 4 *Task-features* and 4 *Cluster-features* as shown in Table 3.

Task features. The task features include the execution time and energy consumption of a task averaged over all PEs, and the earliest start time (EST) and latest finish time (LFT) of that task. The average execution time and energy consumption give the NN a global view of the performance of the task across all PEs. EST and LFT are used to locate the position of the task in the DAG. EST is the earliest time when the task can start its execution, and LFT is the latest time to complete it. Both EST and LFT are calculated by assuming there is an unlimited number of resources:

$$EST_i = w_{i,fast} + \max_{t_j \in \boldsymbol{T}_k^{parent}(i)} (EST_j + c_{j,i}) \tag{10}$$

$$LFT_i = L_{min}(\boldsymbol{T}_k) - \max_{t_j \in \boldsymbol{T}_k^{child}(i)} (LFT_j + w_{j,fast} + c_{i,j}) \tag{11}$$

where $L_{min}(\boldsymbol{T}_k)$ is the minimum makespan of $\boldsymbol{T}_k$ corresponding to unlimited number of resources, $\boldsymbol{T}_k^{parent}(i)$ and $\boldsymbol{T}_k^{child}(i)$ are the set of the parent and child tasks of t_i, $w_{i,fast}$ is the fastest execution time of t_i, and $c_{i,j}$ is the communication time between t_i and t_j.

Cluster features. The cluster features include the execution time and energy consumption of task $t_i \in \mathbf{T}_k$ in a PE in $cluster_n$ and the execution time averaged across all tasks in $\boldsymbol{T}_k$ of a PE in $cluster_n$. Note that if the task cannot run in a specific cluster, we set its execution time to be very long $w_{i,n} = 10 \times D_k^{soft}$, such that the task–cluster allocation probability will always be 0. Another cluster feature is the job-level slack in $cluster_n$, defined as the sum of the job-level slack across PEs in $cluster_n$:

$$SLACK_{k,n}^{cluster} = \sum_{\text{PE}_m \in cluster_n} SLACK_{k,m} \tag{12}$$

Given N clusters, the total number of features is $4{+}4N$. To minimize the runtime overhead of NN, we prune the number of baseline features to one *Task-feature* and two *Cluster-features* using the trainable mask with regularization approach [49]. Thus the total number of features is reduced to $1 + 2N$ after pruning. The performance comparisons using different numbers of features are provided in Section 5.6.

4.2.2. Neural network training

We implemented an optimal scheduling strategy using CP [50] to generate the training dataset. CP is a search-optimization-based method that finds the optimal decision for an optimization objective given multiple constraints. Since the CP algorithm exhaustively searches the decision space, the time to run CP can be several days when there are multiple streaming jobs.

We aim to develop a mechanism for using a single job to generate the dataset representing multiple streaming jobs. We notice that the job-level slack $SLACK_{k,m}$ is a function of the inter-job contentions and can change at runtime. Note that $SLACK_{k,m}$ is also a function of the busy time of PE_m, where $busy_m$ is defined as the time required

Table 3
Input baseline features of NN.

Level	Features	Pruned features
Task	Average execution time of t_i across PEs	
	Average energy consumption of t_i across PEs	
	Latest Finish Time, LFT_i	
	Earliest Start Time, EST_i	√
Cluster	Execution time of t_i	√
	Energy of t_i	
	Average execution time across all tasks in $\boldsymbol{T}_k$	
	Job-level slack for $\boldsymbol{T}_k$ in $cluster_n$	√

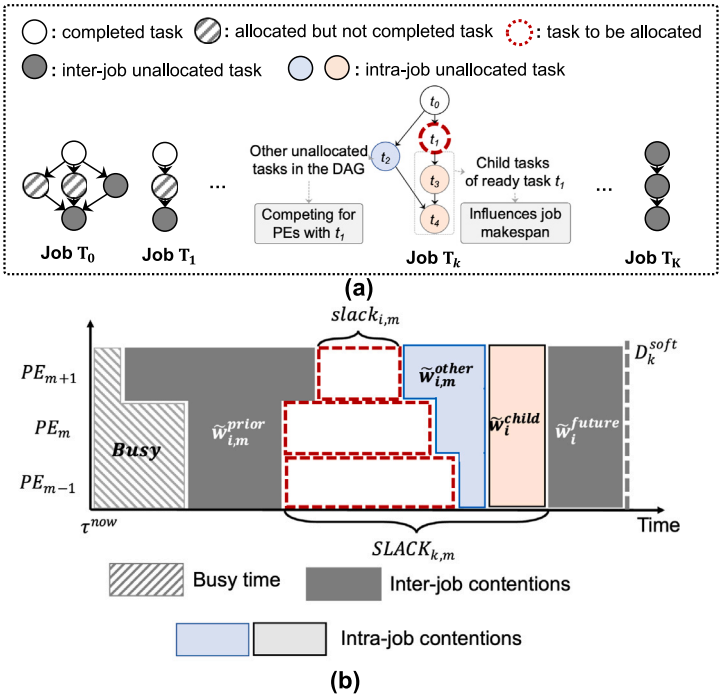


Fig. 4. An example of final allocation when the SoC is running K jobs simultaneously. (a) The task to be scheduled is t_1 (*red* border) in job $\mathbf{T}_k$. (b) Computation of job-level slack for job $\boldsymbol{T}_k$ and task-level slack for $t_i \in \boldsymbol{T}_k$ in different PEs. The job-level slack $SLACK_{k,m}$ in PE_m is derived by subtracting out the time to complete the tasks that have been allocated earlier (busy time) and the time set aside for unallocated tasks from other prior and future jobs (inter-job contentions) from soft deadline. The task-level slack $slack_{i,m}$ is derived from job-level slack by subtracting the time set aside for unallocated tasks from the same job (intra-job contentions).

to complete the allocated tasks in PE_m. Varying $busy_m$ is equivalent to varying $SLACK_{k,m}$. Therefore, to simulate different job-level slack due to inter-job contentions, we vary the busy time of PE_m, to be equal to B_m, where B_m is a variable that is swept from 0 to soft deadline D_k^{soft}.

To minimize the objective function in Eq. (6), the CP scheduler first finds a set of schedules $\mathbf{S_T}$ that minimize the tardiness ($Tardiness$). Then, among these schedules, it chooses the schedule S that minimizes the total energy consumption. The samples are collected from different scenarios obtained by varying the frequency f_n of $cluster_n, n \in [1, N]$ and busy time $busy_m$ of $\text{PE}_m, m \in [1, M]$. The variables f_n and $busy_m$ are randomly sampled as follows:

$$f_n \sim \mathcal{U}(f_n^{min}, \ldots, f_n^{max}), n \in [1, N] \tag{13}$$

$$busy_m \sim \mathcal{U}(0, D_k^{soft}), m \in [1, M] \tag{14}$$

where $\mathcal{U}(\cdot)$ represents uniform distribution, and $f_n^{min}, \ldots, f_n^{max}$ is the set of all supported operating frequencies of $cluster_n$.

Each training sample corresponds to a task and includes its: (a) features which are the corresponding task and cluster features, and (b) the label, which is the cluster ID that the task is allocated to. The training samples are then used to train the NN to learn task-to-PE allocation probability on different PEs in the SoC.

4.3. Final allocation

At time τ^{now}, a ready task $t_i \in \boldsymbol{T}_k$, whose parent tasks have completed execution, is selected to be allocated. The job-level slack $SLACK_{k,m}$ and task-level slack, $slack_{i,m}$, are computed for each candidate PE_m by considering the inter-job and intra-job contentions. If $w_{i,m}$, the execution time of t_i in PE_m, is not greater than $slack_{i,m}$, PE_m is a candidate PE. Finally, t_i is allocated to the most energy-efficient candidate PE. If no PEs can meet the $slack_{i,m}$ requirement, t_i is allocated to the PE with minimum $w_{i,m} - slack_{i,m}$. Computing the task-level slack and identifying the most energy-efficient PE candidate for each task enables *PED* to minimize job tardiness with the least energy consumption.

In the remaining subsection, we introduce the detailed calculation of the job-level and task-level slack. We use the example in Fig. 4 to illustrate these steps.

4.3.1. Job-level slack considering inter-job contentions

The inter-job contentions of $\boldsymbol{T}_k$ come from two types of parallel jobs, jobs arriving before the current time τ^{now} (prior jobs) or later than τ^{now} (future jobs). The unallocated tasks from prior jobs could still be executed in PE_m, thereby reducing the effective time of PE_m to execute $\boldsymbol{T}_k$. The occupancy times in PE_m of the unallocated tasks from prior

jobs, which is the sum of the occupancy time from unallocated tasks in prior jobs:

$$\tilde{w}_{k,m}^{prior} = \sum_{t_i \in T_k^{prior}} \tilde{w}_{i,m} \tag{15}$$

where T_k^{prior} is the set of unallocated tasks from prior jobs, and $\tilde{w}_{i,m}$ is the expected occupancy time computed in Eq. (7). This is shown as a gray box in Fig. 4(b).

During the execution of T_k, new jobs may continue to arrive. If T_k cannot finish before time τ_{k+1} when T_{k+1} arrives, T_k also competes with T_{k+1} for the resources and could result in unnecessary tardiness of T_{k+1}. Therefore to avoid increasing tardiness of future jobs, the occupancy time of future tasks $\tilde{w}_{k,m}^{future}$ should be considered (see the second gray block in Fig. 4(b)). We use the method proposed in [44] to compute the occupancy of future jobs. Estimating the computation of $\tilde{w}_{k,m}^{future}$ involves estimating the arrival times of future jobs before the completion of the current job. These expected arrival times are determined using the historical occurrence of such jobs. The presence of more future jobs leads to longer occupancy times in different PEs. For details, please refer to Section 5.1 in [44].

Eq. (16) summarizes the computation for the job-level slack $SLACK_{k,m}$ in different PEs considering the inter-job contentions:

$$SLACK_{k,m} = D_k^{soft} - avail_m - contention_{k,m}^{inter} \tag{16}$$

where $contention_{k,m}^{inter} = \tilde{w}_{k,m}^{prior} + \tilde{w}_{k,m}^{future}$ is the estimated execution time set aside for both prior and future jobs. PE_m is not available before completing the tasks allocated earlier, therefore, the available time of PE_m is $avail_m = max(\tau^{now}, busy_m)$, which corresponds to the slashed block in Fig. 4(b). If the unallocated tasks in parallel jobs have high allocation probability in a PE, the occupancy times in that PE are high, resulting in a shorter job-level slack for the new job in that PE. Consequently, the tasks in the job will avoid being allocated to that PE.

4.3.2. Task-level slack considering intra-job contentions

To illustrate the computation of the task-level slack, Fig. 4 presents an example of a DAG structure in which task t_1 is the ready task to be scheduled. Task t_0 is the predecessor to task t_1 which has already been completed, but tasks t_2, t_3 and t_4 are yet to be scheduled. We define the child path of task t_1 as a set of tasks in the branch originating from t_1 and ending at the exit task (such as $t_1 \rightarrow t_3 \rightarrow t_4$). The critical child path with the longest expected execution time determines the makespan of the job. While tasks not belonging to the critical child path (such as task t_2) do not determine the makespan, they also compete for the same resources and their occupancy in PEs should be considered. Therefore, the intra-job contentions of task t_i depend on two factors: (1) the *expected execution time of critical child path* of the ready task t_i (the orange block in Fig. 4), and (2) *the occupancy times* in PEs from other tasks in the job T_k (the blue block in Fig. 4).

Expected execution time of critical child path. The expected time to execute every child path is estimated by the allocation probabilities generated in the pre-allocation step and recursively computed by traversing the DAG upward starting from the exit task to t_i as per Eq. (17):

$$\tilde{w}_i^{child} = \max_{t_j \in T_k^{child}(i)} \left(\mathop{\mathbb{E}}_{w_j \sim p_j} [w_j] + \bar{c}_{i,j} + child_path_j \right) \tag{17}$$

where $T_k^{child}(i)$ is the set of the child tasks of t_i and $\bar{c}_{i,j}$ is the average communication cost of t_i and t_j across all PEs. $\mathbb{E}_{w_j \sim p_j}[w_j] = \sum_{m \in M}(p_{j,m} \cdot w_{j,m})$ is the expected execution time of child task t_j computed using its allocation probabilities p_j across all PEs. Note that $\tilde{w}_i^{child}$ is computed only once when a job arrives.

Occupancy due to other tasks. Since the child tasks in the critical child path have already been considered in $\tilde{w}_i^{child}$, we compute the occupancy time in PEs due to other non-critical tasks in $T_k(i)$, which is represented as $T_k^{other}(i)$. The total occupancy time in PE_m due to non-critical tasks is $\tilde{w}_{i,m}^{other} = \sum_{t_j \in T_k^{other}(i)} \tilde{w}_{j,m}$

Task-level slack of t_i. The task-level slack of t_i in PE_m is defined as the difference between the job-level slack and the intra-job contentions:

$$slack_{i,m} = SLACK_{k,m} - contention_{i,m}^{intra} \tag{18}$$

where $contention_{i,m}^{intra} = \tilde{w}_{i,m}^{other} + \tilde{w}_i^{child}$ is the intra-job contentions from all unallocated tasks in T_k.

4.4. Pseudo-code for PED

The PED scheduler is summarized in Algorithm 1. When a new job $\mathbf{T_k}$ arrives, the *Pre_allocation*() step estimates the allocation probability of every task in $\mathbf{T_k}$. When one or more tasks are ready, the *Final_allocation*() step is called to schedule the ready tasks based on their task-level slacks in the order of descending deadlines.

Algorithm 1 PED scheduler

Given: Power consumption of every PE; execution time of executing any tasks on each PE; Offline trained NN $f(\cdot)$;
Input: Status of the currently executing, allocated, and unallocated tasks
Output: Task schedule

```
while True do
    if New job T_k arrives then
        Pre_allocation(T_k)
    end if
    if ready_task_list ≠ ∅ then
        Final_allocation(ready_task_list)
    end if
end while

procedure Pre_allocation(T_k):
    Compute job-level slack SLACK_{k,m} using Eq.(16)
    for t_i ∈ T_k do
        Estimate allocation probability p_{i,m} using Eq.(8) and (9)
    end for
end procedure

procedure Final_allocation(ready_task_list):
    Sort ready_task_list in the descending order of deadline
    for t_i ∈ ready_task_list do
        Compute task-level slack slack_{i,m} using Eq.(15), (16), (17), and (18)
        Add PE_m with slack_{i,m} > w_{i,m} to Candidates
        if Candidate_PE ≠ ∅ then
            Schedule task t_i to the most energy-efficient PE in Candidates
        else
            Schedule task t_i to the PE with minimum w_{i,m} − slack_{i,m}
        end if
    end for
end procedure
```

5. Experimental results

This section first describes the experimental setup that introduces the applications, the SoC configuration and the baseline techniques used for comparison to evaluate the proposed *PED* scheduler. Section 5.2 presents the benefits of *PED* over state-of-the-art baseline techniques for different application soft deadlines. Section 5.3 presents the ablation study to show the effectiveness of the two steps in *PED*. Sections 5.4 and 5.5 illustrate the generalization capabilities of *PED* scheduler across 40 different workloads and different SoC configurations. Section 5.6 presents the effect of pruning the number of NN input features on the allocation probability estimation. Finally, Section 5.7 presents the complexity and overhead analysis of the proposed technique.

5.1. Experimental setup

5.1.1. Applications

We use six real-world applications from the wireless communications and radar systems domain listed in Table 5. The wireless communication domain comprises WiFi and single-carrier (both transmitter and receiver) applications. The temporal mitigation and range detection applications are derived from the radar system domain.

5.1.2. SoC configuration

The SoC configuration includes the Arm big.LITTLE architecture to provide general-purpose computing capabilities and programming flexibility, and hardware accelerators to improve performance and energy efficiency. For wireless communications and radar domain, we choose to include accelerators for (1) fast Fourier transform (FFT), (2) matrix multiplication, and (3) Viterbi decoding, since these three operations consume a significant portion of the total execution time. The SoC configurations are summarized in Table 4. We emphasize that we do not consider power gating since the wake-up delays (in the order of 1–100 ms [51]) are significantly larger than the task execution granularity, which is of the order of microseconds.

5.1.3. Evaluation environment

This work uses the publicly available open-source domain-specific SoC simulator, DS3.[2] Arda et al. [52] DS3 incorporates event-based models to describe the scheduling queues as in a real system. It utilizes the profiled execution time of tasks and detailed analytical thermal and power models proposed by Bhat et al. [53] to estimate the makespan, temperature, power, and energy consumption. To ensure the fidelity of evaluations performed using the simulator with real hardware, DS3 uses execution time and power consumption values measured on commercially available hardware platforms such as the Xilinx Zynq UltraScale+ ZCU102 FPGA [54] and the Samsung Exynos 5422 SoC-based Odroid-XU3 [55]. Since communication between tasks in jobs is substantial, DS3 also uses the communication volumes between tasks analyzed from the job source codes. DS3 estimates the time to move data between PEs using the latencies and bandwidth constraints of the interconnect and the memory.

5.1.4. Constructing workloads and generating application deadlines

To represent streaming application behavior, we construct workloads with a mix of 200 jobs drawn from the 6 domain applications. By default, the number of jobs from every type of application in each workload is the same unless mentioned otherwise. Table 5 presents the number of tasks in each application and the static makespan of one job of each application with the EDF scheduler. The soft deadlines are normalized to the *EDF* makespan $L_{EDF}(\boldsymbol{T}_k)$, when there is only job $\boldsymbol{T}_k$ in the system. The soft deadlines of the 6 applications are varied from tight, represented by $1 \times L_{EDF}(\boldsymbol{T}_k)$, to relaxed, represented by $3 \times L_{EDF}(\boldsymbol{T}_k)$ to thoroughly evaluate the proposed technique.

To simulate dynamic streaming jobs with stochastic arrival, we assume that the injection interval, defined as the inter-arrival time between two consecutive jobs, follows a random distribution with a parameterized mean value. Specifically, we use the exponential distribution since it is commonly used to model the inter-arrival of independent applications in queuing theory [56–58]. We note that simulating streaming jobs with an exponential distribution with a short inter-arrival mean value produces a substantial number of parallel jobs (up to 6 parallel jobs as shown in Fig. 7) executed simultaneously in the SoC.

A shorter injection interval presents higher level of challenge to dynamic task scheduling, since it results in a higher number of parallel jobs and consequently higher PE utilization. This section demonstrates the effectiveness of the proposed *PED* scheduler under different injection intervals. Specifically, we classify the workloads into light, medium and heavy by using average injection intervals of 110 µs, 70 µs and 50 µs, which result in average SoC utilization of 20%, 50% and 70%, respectively. When we extend the experiments to different workload constructions in Section 5.4, the injection intervals are varied such that the average SoC utilization is around 50% for fair comparison.

[2] https://github.com/segemena/DS3.

Table 4
SoC configuration describing the number and type of PEs in each processing cluster and their supported operating frequencies.

Cluster	# of PEs	Frequencies	Description
Arm big	4	0.6–2.0 GHz (0.2 GHz steps)	High-performance energy-intensive
Arm LITTLE	4	0.6–1.4 GHz (0.2 GHz steps)	Low-performance energy-efficient
FFT Acc.	4	1 GHz	32-pt to 2048-pt FFT/IFFT
Viterbi Acc.	2	1 GHz	Viterbi encoding and decoding with $\frac{1}{2}$ rate code
MatMul Acc.	2	1 GHz	Matrix multiplication with fixed size of $(4 \times 64) \times (64 \times 4)$

Table 5
The list of applications from wireless communication and radar system domain and their makespan with the earliest deadline first (EDF) scheduler.

Application	Number of tasks in one job	*EDF* Makespan $L_{EDF}(\boldsymbol{T}_k)$	Deadline
WiFi-TX	27	71 µs	
WiFi-RX	34	301 µs	
Single carrier-TX (SC-TX)	8	56 µs	$1\times - 3\times$
Single carrier-RX (SC-RX)	8	154 µs	$L_{EDF}(\boldsymbol{T}_k)$
Range detection	7	177 µs	
Temporal mitigation	10	81 µs	

5.1.5. Neural network used in pre-allocation

For an SoC with 5 clusters, the NN consists of an input layer with 11 neurons, one hidden layer with 32 neurons and an output layer with 5 neurons (1 per cluster). To train the pre-allocation NN, we generate 46,000 training samples using the CP scheduler (described in Section 4.2.2) by varying the soft deadline, available time of PEs, and cluster frequencies. The collection took around 12 h, and the training of NN took less than 1 min on an Intel i7-9700K CPU.

5.1.6. State-of-the-art techniques used for comparison with PED

The *PED* scheduler is compared with three state-of-the-art scheduling algorithms: the baseline *EDF* scheduler [16] and three deadline-aware energy efficient schedulers *NDES* [1], *PECS* [28], and *PL* [13]. EDF allocates each task to a PE such that the task completes execution as early as possible. When there are several concurrent jobs, the *EDF* scheduler allocates jobs with the earliest deadline first, and within the same job, prioritizes the tasks by their earliest finish time. The *EDF* scheduler only minimizes the job makespan since it focuses on completing a task at the earliest time to meet the deadline. As a consequence, EDF fails to minimize the energy consumption of the jobs in the workload.

NDES [1] first initializes the schedule using HEFT [19], and reschedules the job by gradually stretching the finish time of every task in the initial schedule until the energy is minimized. NDES is a static approach designed for single DAG scheduling, thus it does not consider any inter-job contentions.

PL [13] and *PECS* [17] are two dynamic schedulers for deadline-constrained applications that aim at minimizing energy consumption. Neither of the two methods considers the PE contention from unallocated tasks in parallel jobs. *PL* simply uses the average execution time of

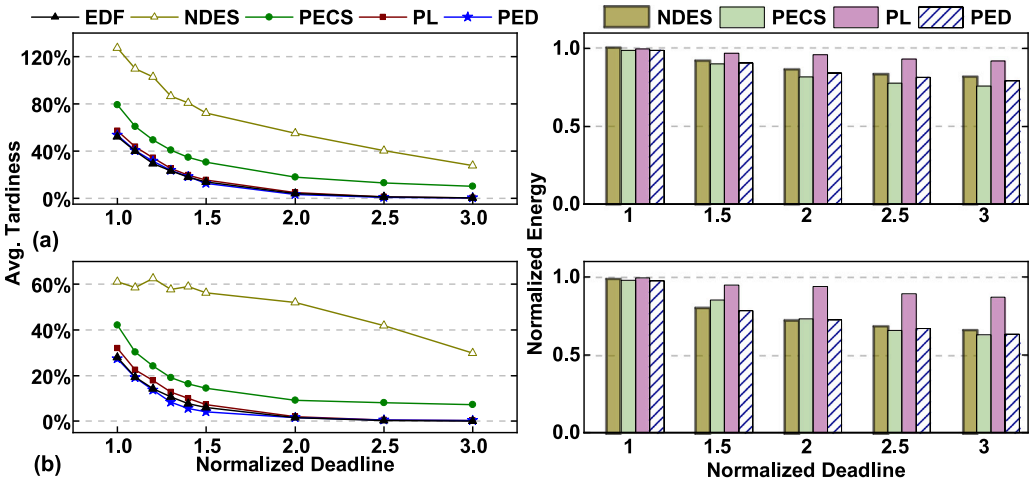


Fig. 5. Comparison of tardiness and normalized energy for *EDF*, *NDES*, *PECS*, *PL* and *PED* schedulers for (a) medium and (b) heavy workloads when the soft deadline is 1.0–3.0×$L_{EDF}(\boldsymbol{T}_k)$.

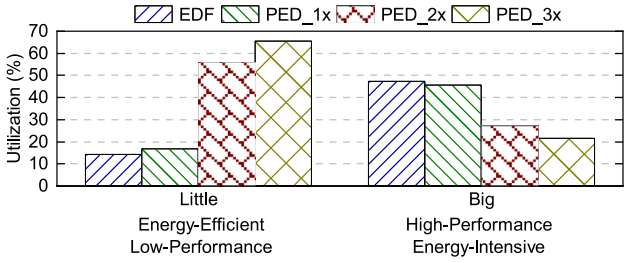


Fig. 6. Utilization of big and LITTLE cluster for *EDF* and *PED* scheduler when the soft deadline is 1×, 2× and $3 \times L_{EDF}(\boldsymbol{T}_k)$ for medium workloads. Note that EDF does not change the scheduling with the soft deadline,.

tasks as the expected execution time for child tasks and *PECS* computes the task-level slack as a proportion of the finish time of every task in the corresponding static HEFT schedule.

As mentioned previously, *EDF* prioritizes meeting the deadlines and does not optimize for energy consumption minimization. As a result, it consumes the highest energy among all the techniques. Hence, we normalize the energy consumption achieved by *PED*, *NDES*, *PECS* and *PL* with *EDF*.

5.2. Evaluation with different soft deadlines and injection intervals

In this section, the big and LITTLE clusters are first set to their maximum frequencies to analyze the benefits of *PED* without using DVFS policies. Then, the scheduler is combined with a deadline-aware DVFS policy *HiLITE* [2].

5.2.1. Evaluation with constant frequency

Fig. 5 shows the utility and energy consumption of *EDF*, *NDES*, *PECS*, *PL* and *PED*, for medium and heavy workloads with 1.0–3.0 × $L_{EDF}(\boldsymbol{T}_k)$ soft deadlines. The proposed *PED* scheduler achieves both the lowest tardiness and the lowest energy consumption. *EDF* scheduler has the same tardiness as *PED* since *EDF* finishes jobs as soon as possible, but it has higher energy consumption. When the soft deadline is tight, like $1 \times L_{EDF}(\boldsymbol{T}_k)$, the available slack to save energy is limited, hence none of the methods can save significant energy. When soft deadline is relaxed, as in $3 \times L_{EDF}(\boldsymbol{T}_k)$, *PED* saves 14% and 22% energy compared to *EDF* for heavy and medium workloads, on average.

The *PL* scheduler has tardiness comparable to *PED*, and its energy consumption is lower than *EDF* but higher than *PED*. Even though the *PL* scheduler allocates tasks to energy-efficient clusters, it uses the average execution time as the expected execution time of child path. Since the average execution time of a task is much longer than the real execution time if the task is supported by accelerators, *PL* is unable to exploit the real task-level slack to minimize energy consumption. Compared with *PL*, the proposed *PED* scheduler can save energy by 10% and 18% on average, for heavy and medium workloads.

The task-level slack of *PECS* is based on the finish time of the static HEFT scheduling results. Thus the energy consumption of *PECS* is comparable with that of *PED*. However, since *PECS* is unaware of the contention from unallocated tasks from concurrent jobs, its tardiness is higher than *PED*. Compared with PECS, the proposed PED scheduler can reduce tardiness by 7.2× and 6.9× on average, for heavy and medium workloads.

Note that the *NDES* scheduler has almost the same energy consumption as *PED* but has >20× higher tardiness compared with proposed *PED*, since *NDES* is a static scheduler and ignores the contention from concurrent jobs. We exclude *NDES* from rest of the comparison, due to its significantly higher tardiness.

The utilization of energy-efficient PEs for *EDF* and *PED* is shown in Fig. 6. It explains why PED achieves higher energy saving compared to *EDF*. *EDF* scheduler runs more tasks in the fast but energy-intensive big cluster to finish jobs early, resulting in high energy consumption. In contrast, *PED* allocates as many tasks as possible to the energy-efficient clusters within the deadline constraint to reduce energy consumption. The utilization of energy-efficient PEs improves from 2% to 51% when the soft deadline increases from 1× to $3 \times L_{EDF}(\boldsymbol{T}_k)$.

Fig. 7 shows the number of concurrent jobs, tardiness and energy consumption as a function of time for a medium workload using *EDF*, *PL*, *PECS* and *PED* schedulers. The bar figure on top shows the average number of concurrent jobs in the system. The number of concurrent jobs is usually around 3 with some busy periods having up to 6 jobs. Note that the PED scheduler always has the same level of tardiness with much lower energy consumption compared with *EDF* and *PL* schedulers, even with a large number of concurrent jobs. The energy consumption of *PED* is almost the same as *PECS*, while the tardiness of *PED* is way lower than *PECS*.

5.2.2. Evaluation with DVFS policy

The energy consumption can be further reduced by turning down the frequency of clusters through DVFS when the workload is light. We consider the DVFS policy, *HiLITE*, an imitation learning-based DVFS policy that maximizes energy efficiency while satisfying soft deadline constraints on embedded SoCs [2].

We combine *HiLITE* with *EDF*, *PECS*, *PL* and the *PED* schedulers and tune the frequencies every 10 ms. The results are as shown in

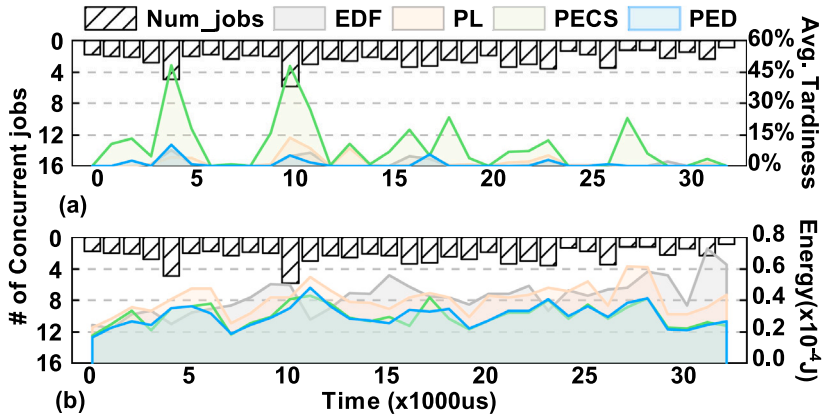


Fig. 7. Relationship between the number of concurrent jobs in the system and the (a) tardiness and (b) energy consumption for *EDF*, *PL*, *PECS* and *PED* schedulers for medium workloads with 2.0×$L_{EDF}(\boldsymbol{T}_k)$ soft deadline.

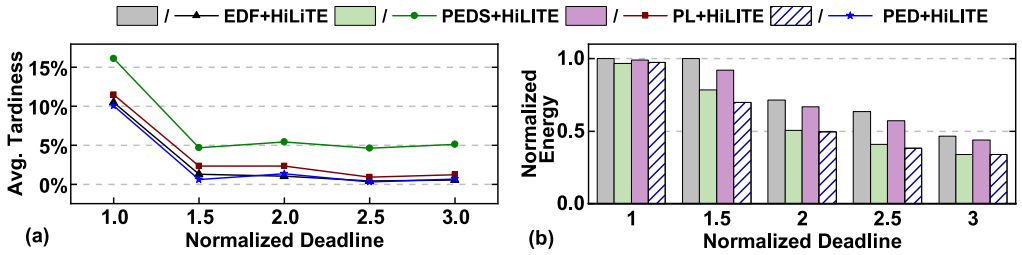


Fig. 8. Comparison of (a) tardiness and (b) normalized energy of *EDF*, *EDF+HiLITE*, *PECS+HiLITE*, *PL+HiLITE* and *PED+HiLITE* for light workloads with 2.0× L_{EDF} soft deadline.

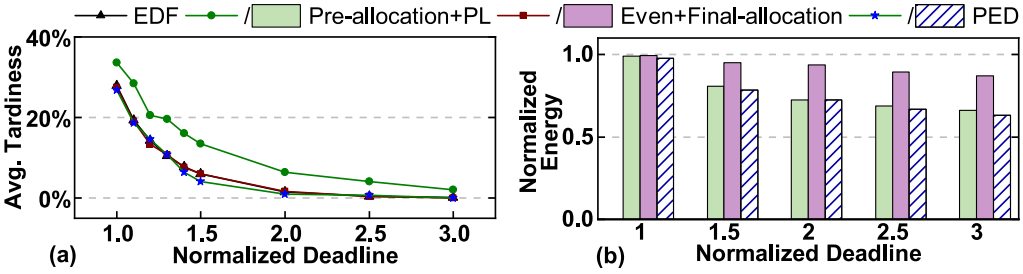


Fig. 9. Comparison of (a) tardiness and (b) normalized energy for *PED* scheduler and its variants for medium workload with 2.0×$L_{EDF}(\boldsymbol{T}_k)$ soft deadline. *Pre-allocation+PL* replaces the final-allocation in *PED* with the task-level slack computation in *PL*. *Even+Final-allocation* replaces the pre-allocation probability in *PED* with random probability.

Fig. 8. Since the DVFS policy only saves energy for light workloads, here we use a light workload where the average SoC utilization is only 20%. *PECS+HiLITE* achieves comparable energy with *PED+HiLITE* but with 7.6× more tardiness. The tardiness of *PED+HiLITE* is not more than that of *EDF+HiLITE* and *PL+HiLITE*, while the energy consumption of *PED+HiLITE* is 19% and 14% less compared to *EDF+HiLITE* and *PL+HiLITE*, on average. Thus, *PED* can run in conjunction with other DVFS policies to further save energy.

5.3. Ablation study

This section evaluates the effectiveness of the pre-allocation and final-allocation steps of *PED*. To evaluate the effectiveness of these two steps, two variants of *PED* are designed, namely, *Even+Final-allocation* and *Pre-allocation+PL*. In *Even+Final-allocation*, the pre-allocation probability obtained using NN-based method is replaced with an evenly distributed probability while keeping the final-allocation step the same as *PED*. On the other hand, *Pre-allocation+PL* keeps the pre-allocation step of *PED* and replaces the available time slack computation in Eq. (18) with the one in *PL* [13], which does not consider the contention from other jobs.

Fig. 9 shows the job tardiness and normalized energy for medium workloads with the soft deadline varying from 1.0× to 3.0× of L_{EDF} $(\boldsymbol{T}_k)$. The proposed *PED* scheduler achieves the lowest energy consumption with comparable tardiness as *EDF*. *Even+Final-allocation* has higher energy consumption than the proposed *PED* since the allocation probability is inaccurate resulting in short task-level slack. Consequently, the tasks cannot be allocated to the slow energy-efficient PEs. Even though *Pre-allocation+PL* has lower energy than *PED*, it has higher tardiness because it overlooks the contention from parallel jobs. The results indicate that the pre-allocation step in *PED* helps in reducing energy consumption, while the final-allocation step minimizes job tardiness.

5.4. Evaluation with different workloads

The experimental evaluations presented until this section consider the same workload, i.e., the number and type of jobs that constitute the workload are the same in all scenarios. To demonstrate the generalization of *PED* across different compositions of workloads, we evaluate the performance of *PED* on 40 different workloads with different ratios of constituent jobs. Each of the workloads is dominated by one of WiFi-TX, WiFi-RX, range detection, SC-TX, SC-RX, and temporal mitigation. We

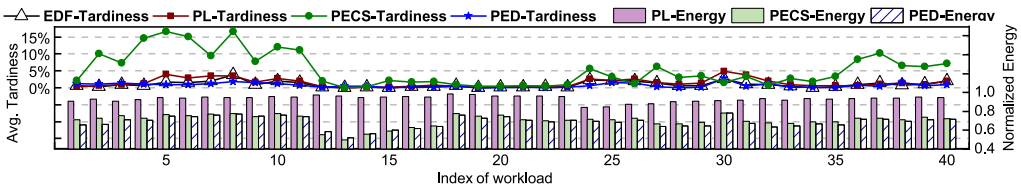


Fig. 10. Tardiness and energy of *PL*, *PECS* and *PED* normalized to *EDF* results for different workloads. The average injection interval is set for medium. The workloads 1–5 are dominated by WiFi-TX, 6–11 by WiFi-RX, 12–17 by range detection, 18–23 by SC-TX, 24–29 by SC-RX, and 30–35 by temporal mitigation. In the last 5 workloads (36–40), the 6 applications are distributed evenly.

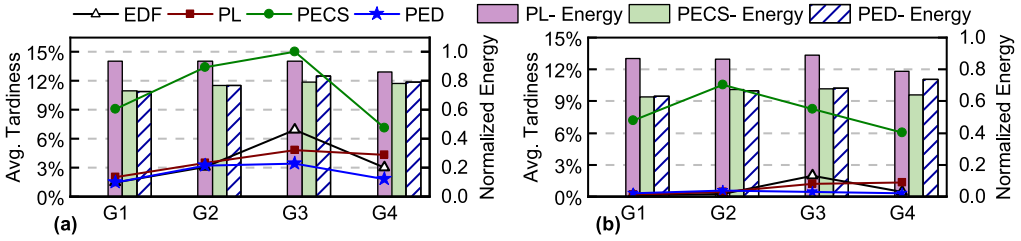


Fig. 11. Tardiness and normalized energy of *EDF*, *PL*, *PECS* and *PED* with G1, G2, G3 and G4 SoC configurations for medium workloads with (a) 2.0× and (b) 3.0× soft deadline.

Table 6
Configuration of different SoC configurations.

SoC Config.	LITTLE PEs	Big PEs	FFT PEs	Viterbi PEs	MatMul PEs
G1	4	4	4	2	2
G2	2	2	2	2	2
G3	1	1	1	1	1
G4	4	4	0	2	0

also consider 5 workloads in which the jobs from 6 domain applications are distributed evenly. The average SoC utilization is around 50%, and the soft and hard deadlines are set to $2.0 \times L_{EDF}(\boldsymbol{T}_k)$, respectively. Fig. 10 shows the tardiness and normalized energy of *EDF*, *PL*, *PECS* and *PED* schedulers. For various workload combinations, *PED* scheduler consistently achieves the lowest tardiness, which is 10.2× less than *PECS* on average. And *PED* scheduler saves energy by 31% and 24%, on average, compared to *EDF* and *PL* schedulers with comparable tardiness.

5.5. Evaluation with different SoC configurations

This section presents the performance of the proposed *PED* scheduler for different SoC configurations and the corresponding comparisons to *EDF*, *PL* and *PECS*. The SoC configurations vary in the number of general-purpose cores, and hardware accelerators. Since the largest SoC configuration supported by DS3 is G1, we include G1 and smaller configurations G2, G3 and G4 in this evaluation. Table 6 includes the details of the four configurations. Fig. 11(a) and (b) show the comparison of tardiness and energy between *EDF*, *PL*, *PECS* and *PED* schedulers for medium workloads with 2× and 3× $L_{EDF}(\boldsymbol{T}_k)$, respectively. Compared with *PECS*, the tardiness of *PED* scheduler is reduced by 11.4×, while achieving comparable energy consumption (2% difference on average). The *PED* scheduler can reduce 25% and 14% of energy, on average, for various SoCs when compared with *EDF* and *PL*, respectively, with comparable or even lower tardiness.

5.6. Effect of feature pruning in NN

In Section 4.2, the input features of NN are pruned from 24 baseline features (4 for task+4 per cluster × 5 clusters = 24) to 11 features (1 for task+ 2 per cluster × 5 clusters = 11). We tune the hyper-parameters of the feature pruning algorithm to further reduce the number of features to 6 (1 for task +1 per cluster ×5 clusters = 6) with only the task execution time as the cluster feature. Fig. 12 shows the results of *PED* scheduler using NN with 24, 11 and 6 features. *PED* with 11 features achieves the same tardiness and energy compared to *PED* with all 24 features. With only 6 features, *PED* still achieves comparable tardiness and energy to *PED* with all 24 features for G1 configuration. However, for smaller configurations such as G2, G3, and G4, if only 6 features are used, the energy is 4% higher than *PED* with 24 features for $2\times L_{EDF}(\boldsymbol{T}_k)$ soft deadline. The gap grows to >10% with longer soft deadline of $3 \times L_{EDF}(\boldsymbol{T}_k)$, as shown in Fig. 11(b). So we conclude that 11 features, selected by the pruning method, are necessary to achieve satisfactory tardiness and energy consumption.

5.7. Asymptotic complexity and overhead analysis

This section analyzes the computational complexity of *PED* for scheduling a job with T tasks when the number of PEs is M.

- The complexity of computing allocation probability using NN-based pre-allocation is $O(MT)$.
- The job-level slack $SLACK_{k,m}$ is a function of all unallocated tasks from prior jobs in all PEs, hence it has a complexity of $O(MT_{prior})$, where T_{prior} is the number of unallocated tasks from prior jobs and usually less than T in our experiments.
- The complexity of prioritizing the ready tasks based on the deadline is $O(T \cdot T_{ready})$, where T_{ready} is the number of ready tasks.
- To compute the task-level slack of the ready task with the highest priority, the scheduler has to calculate the occupancy time of remaining tasks in the job in M PEs. Hence its complexity is $O(MT^2)$.

The overall complexity for allocating a job with T tasks to M PEs using *PED*, is $O(MT^2)$.

PED scheduler has two steps, pre-allocation and final-allocation. We implemented both steps on the Nvidia Jetson Xavier NX hardware platform. They are profiled on the Arm-based Denver cores operating at a frequency of 1.9 GHz. On hardware, the measured latency of the pre-allocation with 11 input features is 360 ns. The measured time overhead of the final-allocation on hardware is 500 ns.

We note that the NN is invoked T times for an application job with T unique tasks. Hence the runtime overhead of pre-allocation is in the critical path of the execution of the task. In contrast, the final-allocation step of every task is not in the critical path since the final-allocation can run simultaneously with the execution of its parent tasks. Thus

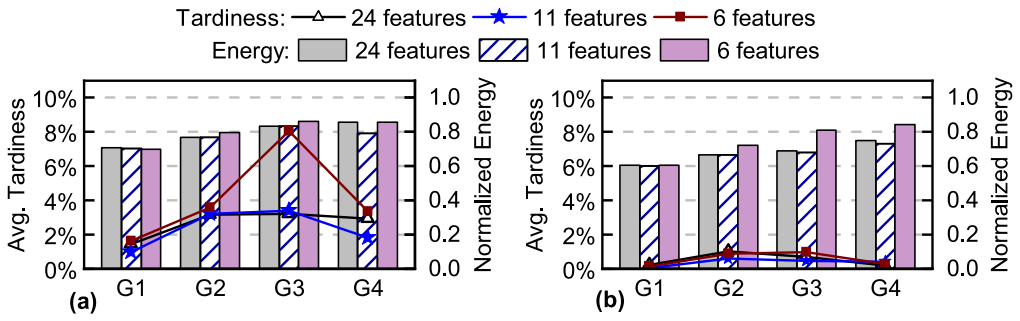


Fig. 12. Tardiness and normalized energy of *PED* using NN with 24, 11 and 6 features for G1, G2, G3 and G4 SoC configurations with medium workloads and (a) 2.0× and (b) 3.0× soft deadline.

the overhead of final-allocation can be hidden for all tasks except the entry task. The overhead of PED for WiFi-TX, WiFi-RX, SC-TX, SC-RX, Range Detection and Temporal Mitigation in our experiments is 0.8%, 0.3%, 1.1%, 0.5%, 0.3% and 0.9% of the job makespan, respectively. Furthermore, the memory overhead to store the weights of (3 layers, 16 neurons in hidden layer, 11 inputs and 5 outputs) is 2 KB. Thus the NN-based PED scheduler has low overhead in terms of timing and memory footprint.

6. Conclusion

Heterogeneous SoCs that integrate general-purpose processors and hardware accelerators provide superior performance and energy efficiency over homogeneous SoCs. Jobs with soft deadline constraints, such as those in communications, can tolerate missing deadlines at the expense of lower QoS, measured by tardiness. This paper presents a probabilistic energy-efficient deadline-aware scheduler, *PED*, to minimize job tardiness with minimum energy consumption in heterogeneous SoCs. The main novelty of our approach lies in *(i)* building a NN to estimate the task-to-PE allocation probability and *(ii)* deriving task-level slack by considering the inter-job and intra-job contentions. The performance of the proposed *PED* scheduler is evaluated with six communication and radar application domain jobs on an SoC with 4 Arm big.LITTLE cores and 3 types of hardware accelerators. The results show that when big.LITTLE cores run at maximum frequency, *PED* can save energy by 22% and 18% without increasing the tardiness compared to *EDF* and *PL* schedulers, respectively. When compared with *PECS* and *NDES* schedulers, *PED* reduces the tardiness by 6.9× and >20× on average, respectively. Furthermore, *PED* achieves 19% and 14% lower energy consumption than the *EDF* and *PL* schedulers with comparable tardiness, and 7.6× lower tardiness than *PECS* scheduler with comparable energy consumption, when combined with a state-of-the-art DVFS policy. Thus, *PED* effectively minimizes the tardiness with minimum energy consumption for streaming jobs in heterogeneous SoCs.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Xing Chen reports financial support was provided by Defense Advanced Research Projects Agency. Xing Chen reports financial support was provided by Air Force Research Laboratory.

Data availability

Data will be made available on request.

Acknowledgment

This material is based on research sponsored by Air Force Research Laboratory (AFRL) and Defense Advanced Research Projects Agency (DARPA) under agreement number FA8650-18-2-7860. The U.S. Government is authorized to reproduce and distribute reprints for Governmental purposes notwithstanding any copyright notation thereon. The views and conclusion contained herein are those of the authors and should not be interpreted as necessarily representing the official policies or endorsements, either expressed or implied, of Air Force Research Laboratory (AFRL) and Defense Advanced Research Projects Agency (DARPA) or the U.S. Government.

References

[1] G. Xie, G. Zeng, X. Xiao, R. Li, K. Li, Energy-efficient scheduling algorithms for real-time parallel, IEEE Trans. Parallel Distrib. Syst. 28 (12) (2017) 3426–3442.
[2] A.L. Sartor, A. Krishnakumar, S.E. Arda, U.Y. Ogras, R. Marculescu, Hilite: Hierarchical and lightweight imitation learning for power, IEEE Comput. Archit. Lett. 19 (1) (2020) 63–67.
[3] D. Green, et al., Heterogeneous Integration at DARPA: Pathfinding and Progress in, ECTC, May, 2018.
[4] S. Tang, S. Voronov, J.H. Anderson, Extending EDF for soft real-time scheduling on unrelated multiprocessors, in: 2021 IEEE Real-Time Systems Symposium (RTSS), 2021, pp. 253–265, http://dx.doi.org/10.1109/RTSS52674.2021.00032.
[5] S.K. Lee, P.N. Whatmough, M. Donato, G.G. Ko, D. Brooks, G.-Y. Wei, SMIV: A 16-nm 25-mm^2 SoC for IoT with Arm Cortex-A53, and coherent accelerators, IEEE J. Solid-State Circuits 57 (2) (2021) 639–650.
[6] H. Kooti, D. Mishra, E. Bozorgzadeh, Reconfiguration-aware real-time scheduling under QoS constraint, in: 16th Asia and South Pacific Design Automation Conference (ASP-DAC 2011), IEEE, 2011, pp. 141–146.
[7] M. Shaghaghi, R.S. Adve, Task selection and scheduling in multifunction multi-channel radars, in: 2017 IEEE Radar Conference (RadarConf), IEEE, 2017, pp. 0969–0974.
[8] M. Shaghaghi, R.S. Adve, Machine learning based cognitive radar resource management, in: 2018 IEEE Radar Conference (RadarConf18), IEEE, 2018, pp. 1433–1438.
[9] S. Li, M. Song, P.-J. Wan, S. Ren, A 2-approximation algorithm for scheduling parallel and time-sensitive, IEEE Trans. Parallel Distrib. Syst. 27 (7) (2015) 1864–1878.
[10] B. Khemka, et al., Utility functions and resource management in an oversubscribed, IEEE Trans. Comput. 64 (8) (2014) 2394–2407.
[11] B.P. Rimal, M. Maier, Workflow scheduling in multi-tenant cloud computing environments, IEEE Trans. Parallel Distrib. Syst. 28 (1) (2016) 290–304.
[12] Z. Yang, H. Wu, Y. Xu, Y. Wu, H. Zhong, W. Zhang, Hydra: Deadline-aware and efficiency-oriented scheduling for deep learning jobs on heterogeneous GPUs, IEEE Trans. Comput. 72 (8) (2023) 2224–2236.
[13] M. Safari, R. Khorsand, PL-DVFS: Combining power-aware list-based scheduling algorithm, J. Supercomput. 74 (10) (2018) 5578–5600.
[14] D. Senapati, A. Sarkar, C. Karfa, PRESTO: A penalty-aware real-time scheduler for task graphs on, IEEE Trans. Comput. 71 (2) (2021) 421–435.
[15] V. Arabnejad, K. Bubendorfer, B. Ng, Budget and deadline aware E-science workflow scheduling in clouds, IEEE Trans. Parallel Distrib. Syst. 30 (1) (2018) 29–44.
[16] M. Kargahi, A. Movaghar, Non-preemptive earliest-deadline-first scheduling policy: A, in: IEEE International Symposium on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems, 2005, pp. 201–208.

[17] S. Tian, W. Ren, Q. Deng, S. Zou, Y. Li, A predictive energy consumption scheduling algorithm for, IEEE Trans. Green Commun. Netw. 6 (2) (2022) 979–991.
[18] Z. Tang, et al., An energy-efficient task scheduling algorithm in DVFS-enabled cloud, J. Grid Comput. 14 (1) (2016) 55–74.
[19] H. Topcuoglu, S. Hariri, M.-Y. Wu, Performance-effective and low-complexity task scheduling for, IEEE Trans. Parallel Distrib. Syst. 13 (3) (2002) 260–274.
[20] J.D. Ullman, NP-complete scheduling problems, J. Comput. System Sci. 10 (3) (1975) 384–393.
[21] H. Arabnejad, J.G. Barbosa, List scheduling algorithm for heterogeneous systems by an optimistic cost table, IEEE Trans. Parallel Distrib. Syst. 25 (3) (2013) 682–694.
[22] D. Senapati, A. Sarkar, C. Karfa, HMDS: A makespan minimizing DAG scheduler for heterogeneous distributed systems, ACM Trans. Embedded Comput. Syst. (TECS) 20 (5s) (2021) 1–26.
[23] J. Blythe, et al., Task scheduling strategies for workflow-based applications in grids, in: IEEE International Symposium on Cluster Computing and the Grid, vol. 2, 2005, pp. 759–767.
[24] H. Arabnejad, J. Barbosa, Fairness resource sharing for dynamic workflow scheduling on heterogeneous systems, in: 2012 IEEE 10th International Symposium on Parallel and Distributed Processing with Applications, IEEE, 2012, pp. 633–639.
[25] H. Zhao, R. Sakellariou, Scheduling multiple DAGs onto heterogeneous systems, in: Proceedings 20th IEEE International Parallel & Distributed Processing Symposium, IEEE, 2006, pp. 14–27.
[26] Q. Huang, et al., Enhanced energy-efficient scheduling for parallel applications in, in: 12th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing, 2012, pp. 781–786.
[27] Z. Li, et al., Cost and energy aware scheduling algorithm for scientific workflows, IEEE Trans. Serv. Comput. 11 (4) (2015) 713–726.
[28] J. Huang, R. Li, J. An, H. Zeng, W. Chang, A DVFS-weakly-dependent energy-efficient scheduling approach for, IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst. 40 (12) (2021) 2481–2494.
[29] G. Xie, J. Jiang, Y. Liu, R. Li, K. Li, Minimizing energy consumption of real-time parallel applications using, IEEE Trans. Ind. Inform. 13 (3) (2017) 1068–1078.
[30] M. Żotkiewicz, M. Guzek, D. Kliazovich, P. Bouvry, Minimum dependencies energy-efficient scheduling in data centers, IEEE Trans. Parallel Distrib. Syst. 27 (12) (2016) 3561–3574.
[31] A. Mascitti, T. Cucinotta, Dynamic partitioned scheduling of real-time tasks on ARM big. LITTLE architectures, in: 29th International Conference on Real-Time Networks and Systems, 2021, pp. 1–11.
[32] S. Sahoo, B. Sahoo, A.K. Turuk, A learning automata-based scheduling for deadline sensitive task in, IEEE Trans. Serv. Comput. 14 (6) (2019) 1687–1699.
[33] A. Goens, et al., Tetris: A multi-application run-time system for predictable execution, in: Proceedings of the 20th International Workshop on Software and Compilers for Embedded Systems, 2017, pp. 11–20.
[34] R. Khasanov, J. Castrillon, Energy-efficient runtime resource management for adaptable, in: 2020 Design, Automation & Test in Europe Conference & Exhibition (DATE), 2020, pp. 909–914.
[35] S. Moulik, Z. Das, R. Devaraj, S. Chakraborty, SEAMERS: A semi-partitioned energy-aware scheduler for heterogeneous multicore real-time systems, J. Syst. Archit. 114 (2021) 101953.
[36] Y. Sharma, S. Chakraborty, S. Moulik, ETA-HP: an energy and temperature-aware real-time scheduler for heterogeneous platforms, J. Supercomput. 78 (8) (2022) 1–25.
[37] S. Moulik, R. Chaudhary, Z. Das, A. Sarkar, EA-HRT: An energy-aware scheduler for heterogeneous real-time systems, in: 2020 25th Asia and South Pacific Design Automation Conference (ASP-DAC), IEEE, 2020, pp. 500–505.
[38] S. Moulik, Z. Das, G. Saikia, CEAT: a cluster based energy aware scheduler for real-time heterogeneous systems, in: 2020 IEEE International Conference on Systems, Man, and Cybernetics (SMC), IEEE, 2020, pp. 1815–1821.
[39] Y. Sharma, S. Moulik, FATS-2TC: A fault tolerant real-time scheduler for energy and temperature aware heterogeneous platforms with two types of cores, Microprocess. Microsyst. 96 (2023) 104744.
[40] Y. Sharma, S. Moulik, CETAS: a cluster based energy and temperature efficient real-time scheduler for heterogeneous platforms, in: Proceedings of the 37th ACM/SIGAPP Symposium on Applied Computing, 2022, pp. 501–509.
[41] L. Yin, J. Zhou, J. Sun, A stochastic algorithm for scheduling bag-of-tasks applications on hybrid clouds under task duration variations, J. Syst. Softw. 184 (2022) 111123.
[42] J. Zhou, T. Wei, Stochastic thermal-aware real-time task scheduling with considerations of soft errors, J. Syst. Softw. 102 (2015) 123–133.
[43] X. Tang, W. Cao, H. Tang, T. Deng, J. Mei, Y. Liu, C. Shi, M. Xia, Z. Zeng, Cost-efficient workflow scheduling algorithm for applications with deadline constraint on heterogeneous clouds, IEEE Trans. Parallel Distrib. Syst. 33 (9) (2021) 2079–2092.
[44] X. Chen, U. Ogras, C. Chakrabarti, Probabilistic risk-aware scheduling with deadline constraint for, ACM Trans. Embedded Comput. Syst. (TECS) 21 (2) (2022) 1–27.
[45] H. Sultan, G. Ananthanarayanan, S.R. Sarangi, Processor power estimation techniques: A survey, Int. J. High Perform. Syst. Archit. 5 (2) (2014) 93–114.
[46] PerfWiki, Perf Wiki, URL https://perf.wiki.kernel.org/index.php/Main_Page.
[47] K. London, S. Moore, P. Mucci, K. Seymour, R. Luczak, The papi cross-platform interface to hardware performance counters, in: Department of Defense Users' Group Conference Proceedings, 2001, pp. 18–21.
[48] U. Gupta, C.A. Patil, G. Bhat, P. Mishra, U.Y. Ogras, Dypo: Dynamic pareto-optimal configuration selection for, ACM Trans. Embedded Comput. Syst. (TECS) 16 (5s) (2017) 1–20.
[49] F. Nie, H. Huang, X. Cai, C. Ding, Efficient and robust feature selection via joint L1, 1-norms, Proc. Adv. Neural Inf. Process. Syst. (NIPS) (2010) 1813–1821.
[50] IBM, V12.8: User's manual for CPLEX, 46, (53) 2009, p. 157,
[51] N. Madan, A. Buyuktosunoglu, P. Bose, M. Annavaram, A case for guarded power gating for multi-core processors, in: 2011 IEEE 17th International Symposium on High Performance Computer Architecture, IEEE, 2011, pp. 291–300.
[52] S.E. Arda, A. Krishnakumar, A.A. Goksoy, N. Kumbhare, J. Mack, A.L. Sartor, A. Akoglu, R. Marculescu, U.Y. Ogras, DS3: A system-level domain-specific system-on-chip simulation framework, IEEE Trans. Comput. 69 (8) (2020) 1248–1262.
[53] G. Bhat, G. Singla, A.K. Unver, U.Y. Ogras, Algorithmic optimization of thermal and power management for heterogeneous mobile platforms, IEEE Trans. Very Large Scale Integr. (VLSI) Syst. 26 (3) (2017) 544–557.
[54] Zynq, Zynq ZCU102 Evaluation Kit, URL https://www.xilinx.com/products/boards-and-kits/ek-u1-zcu102-g.html.
[55] Hardkernel, ODROID-XU3. URL https://wiki.odroid.com/old_product/odroid-xu3/odroid-xu3.
[56] G.L. Stavrinides, H.D. Karatza, Scheduling different types of applications in a SaaS cloud, in: Proceedings of the 6th International Symposium on Business Modeling and Software Design (BMSD'16), 2016, pp. 144–151.
[57] V. Arabnejad, K. Bubendorfer, B. Ng, Dynamic multi-workflow scheduling: A deadline and cost-aware approach for commercial clouds, Future Gener. Comput. Syst. 100 (2019) 98–108.
[58] D. Gross, Fundamentals of Queueing Theory, John Wiley & Sons, 2008.